Source: docs\REPORT_INDEX_AUDIT.md | Export: 11 November 2026

Reporting index follow-up - 22 October 2026

Confirmed outcome

All seven owner-created orders indexes are READY. The continued failure is an aggregate-field ordering mismatch, not an unfinished build and not a need to recreate those seven indexes. Four additional exact definitions are required to support the deployed dashboard with and without dates. The earlier CATALOG_RECOVERY.md aggregate index advice was incorrect: it listed reportingPaise before collectedPaise/refundPaise. The backend recovery links require collectedPaise, refundPaise, reportingPaise in that order.

No indexes/rules were published, no production records were modified, and no commit/push/deployment occurred. Production access was read-only: index metadata, public bundles, explain-only plans and four reporting aggregate reads (repeated once with the exact deployed filter-builder order). All aggregates failed before returning results; no order values, customer records or tokens were retained in the evidence.

Deployed version and access limits

- Owner-provided Production Current deployment: https://parinayasilks-4bf2k7zrl-parinaya-silks.vercel.app, branch main, commit a5972cc (Customer, Administrator, Storefront). Source inspection used git show a5972cc:server/business.ts, not the uncommitted local replacement.
- Public production alias returned /assets/index-CZQmKhMp.js (SHA256 1d4d6f7cc45cfed20559fa5ecb6bd55a6d07bc78c0a7640625b04650df165918) and AdminAccess-twVgmmdl.js. Its older Products & Inventory sidebar is consistent with the supplied screenshot; the new Customer Enquiries sidebar is absent. Frontend assets alone do not prove backend identity.
- Independent Vercel deployment/source retrieval was blocked: the existing local CLI login received HTTP 403 Not authorized for deployment, alias and team reads. The immutable deployment redirects to vercel.com/login. The commit binding is therefore from the owner-provided deployment metadata, not an independently downloaded server artifact. No credentials were requested or changed.

The reports function at a5972cc creates a base orders collection query with optional inclusive createdAt bounds, then:

1. Seven aggregates filtered by status == each recognized order state.
2. One aggregate filtered by paymentStatus == due and status IN Placed/Confirmed/Processing/Shipped/Delivered.

Each aggregate computes count plus sums of reportingPaise, collectedPaise and refundPaise. Total order value/collections/refunds are reduced from the per-status results; there is no unfiltered or payment-only aggregate call. The four preserved earlier definitions (no status prefix, with optional paymentStatus/date) are unused in both this deployed commit and local code. Do not add them as a remedy. Local pending code keeps these four aggregate query shapes and adds read-only snapshot/count completeness checks and safe query labels; its three independent count shapes already plan successfully against the READY indexes.

Actual READY metadata

Target verified through the supplied local service account: project parinaya-silks, database (default). Metadata read 2026-10-22T15:15:29.169Z (22 October 2026, 20:45:29 IST). Every row below has COLLECTION scope; all fields are ASCENDING.

Index ID	Fields in order	State
CICAgOjXh4EJ	status, paymentStatus, reportingPaise, collectedPaise, refundPaise, **name**	READY
CICAgJim14AK	status, reportingPaise, collectedPaise, refundPaise, **name**	READY
CICAgJj7z4EK	status, paymentStatus, createdAt, reportingPaise, collectedPaise, refundPaise, **name**	READY
CICAgJjF9oIK	status, createdAt, reportingPaise, collectedPaise, refundPaise, **name**	READY
CICAgNi47oMK	status, createdAt, **name**	READY
CICAgJiH2JAK	status, paymentStatus, createdAt, **name**	READY
CICAgOjXh4EL	status, paymentStatus, **name**	READY

Four READY definitions put the sum fields in the earlier incorrect order. The remaining three are independent count indexes; those counts succeeded in live explain-only checks. None should be recreated. Metadata responses also included product entries; those were excluded by their full collectionGroup path before evaluating orders indexes.

Minimal additional indexes - direct backend evidence

Every row: collection orders, queryScope COLLECTION, every listed field ASCENDING, no array modes. Fields are in the exact sequence decoded from Firestore create_composite protobuf links. **name** is the final ascending document-path tie-break.

Query	Exact fields	Backend creation link
Per-status without dates	status, collectedPaise, refundPaise, reportingPaise, **name**	Exact per-status without dates index
Per-status with dates	status, createdAt, collectedPaise, refundPaise, reportingPaise, **name**	Exact per-status with dates index
COD due without dates	paymentStatus, status, collectedPaise, refundPaise, reportingPaise, **name**	Exact cod due without dates index
COD due with dates	paymentStatus, status, createdAt, collectedPaise, refundPaise, reportingPaise, **name**	Exact cod due with dates index

For the screenshot with 1-22 October dates, the dated per-status index is the first blocker; after that succeeds, the dated COD-due index is also required. The two undated variants support initial dashboard loading and clearing the date filters. Four definitions are therefore the smallest complete set for the existing workflow.

All four shapes were reproduced using the same Admin SDK dependency, explicit project/database and predicates from the deployed commit. The dated reproduction used inclusive 2026-10-01 00:00:00 +05:30 through 2026-10-22 23:59:59.999 +05:30. A representative Placed status exercises the shared equality shape used by all seven states. Each aggregate read returned code 9 plus its exact creation URL above. Changing only the filter-builder call order to match the deployed function reproduced identical links. No guessed field ordering or unused-query indexes are included.

Explain-only aggregate failures reported only no matching index found, without recovery links. The subsequent ordinary read-only aggregate calls supplied the authoritative creation links. The encoded google.firestore.admin.v1.Index objects were decoded using the installed Firestore protobuf definitions and compared with the actual READY metadata. All four requested definitions are absent. The existing reportingPaise-first definitions are not equivalent in their aggregate suffix ordering.

Only these four additional definitions are proposed for future owner publication. They are already appended locally to firestore.indexes.json; all prior definitions and fieldOverrides remain. No publication was performed. Hosted dashboard success and the readiness of these four new definitions remain unverified because they have not been created by this task.

Local changes and regression evidence

- firestore.indexes.json: append only the four backend-requested canonical definitions.
- scripts/fixtures/report-index-evidence.json: sanitized live metadata plus exact missing-index links/decoded definitions and owner-reported commit. No credentials, request headers, order values or customer information.
- scripts/catalog-indexes.test.ts: correct the earlier audit assumption; compare actual report-builder shapes to the backend-derived fixture, decode every real link, verify all seven snapshot indexes were READY and assert the four requested definitions were absent. Preserve browse/count coverage. This regression fails if only the previous wrong-order definitions exist.
- server/errors.ts, server/business.ts, server/diagnostics.ts: report failures now carry a static query-shape label (status/cod-due, aggregate/count, all-dates/date-range) through the existing safe error boundary. Original codes and trusted index links remain available server-side. Public responses do not expose query labels/links. No raw dates, bodies, status values or customer information are logged.
- scripts/catalog-handler.test.ts: verify the exported HTTP handler preserves a wrapped report index failure as safe 503/index-required with correlated server diagnostics.

Validation results are recorded after checks finish. Previous customer/admin/browser regressions remain documented in CATALOG_RECOVERY.md; no frontend behavior was changed by this follow-up. Emulator success alone cannot establish production index coverage; the missing-index evidence here comes from live read-only Firestore responses.

Official interface references: Firestore ListIndexes and Vercel deployment file access.

Validation completed for this follow-up

- Typecheck: pass.

- Unit suite: 73/73 pass, including the four real-link decode/missing-definition regression, report-builder/index coverage, safe wrapper diagnostics and exported handler mapping.
- Production frontend build: pass (existing chunk-size advisory only).
- Local COD/order and business API emulator suites: both pass on the updated server code. These cover authorization/isolation, positive collection/refund report totals, date validation, missing-field rejection and catalog regression behavior.
- Packaged Vercel function build/invocation: pass using the pinned local builder and existing runtime harness. This is local packaging/runtime evidence, not a deployed artifact download.
- Existing browser suites were not repeated in this follow-up because no frontend code changed; the preceding task recorded 11 passing browser suites. Browser and emulator success were not used to infer production index readiness.
- git diff --check: pass. Existing uncommitted work was preserved. The only new index definitions in this follow-up are the four exact missing definitions; none of the seven READY definitions was removed or recreated.

Remaining boundary: the deployed server artifact could not be fetched independently due to Vercel access restrictions. The source comparison is bound to the exact Production Current commit supplied by the owner. The four live missing-index responses nevertheless establish the precise missing combinations for those deployed source queries. No additional generic index setup, user credentials, environment changes or unused historical indexes are recommended.